

İçindekiler

Teknik Sunum — Anlatım Planı

1. Sunum akışı — hangi sırayla anlatacaksın
2. Hangi bölüm rehberin neresinden anlatılıyor
3. ★ Bir iş emrinin hayatı
4. Bilinen teknik borçlar
5. Muhtemel sorular ve hazır cevaplar
6. Anlatım sırasında dikkat
7. Rehber güncellenirken (kendime not)

Teknik Sunum — Anlatım Planı

Bu dosya içerik değil, plan. Anlatılacak her şeyin kaynağı `proje-teknoloji-ve-plan.md` dosyasıdır (~190 sayfa). Burada yalnızca *hangi sırayla*, *ne kadar sürede* ve *nasıl* anlatılacağı duruyor.

Aynı bilgi iki dosyada tutulmuyor: bir konunun gerekçesi değiştiğinde tek yer güncelleniyor.

1. Sunum akışı — hangi sırayla anlatacaksın

Yaklaşık 25–30 dakikalık bir anlatım için sıra:

#	Bölüm	Süre	Amacı
1	Proje ve karar çerçevesi	2 dk	"Neye göre karar verdim" — geri kalan her şeyin zemini
2	Stack çevirisi (.NET → JS)	4 dk	En çok soru gelecek yer, baştan açıklığa kavuşsun
3	Mimari ve katmanlar	4 dk	Bağımlılık yönü ve nasıl zorlandığı
4	Veri modeli	3 dk	Tablolar, ilişkiler, index kararları
5	Bir iş emrinin hayatı (sunucu)	5 dk	★ Sunumun kalbi — sistemi baştan sona canlı gösterir
5b	Bir ekranın hayatı (arayüz/UI)	3 dk	Aynı işi frontend için yapar — BÖLÜM G
6	Zor maddeler	6 dk	Factory, durum makinesi, transaction, arka plan işleri, DI
7	Test ve teslim	3 dk	Kalite kapıları ve Docker
8	Teknik borçlar	2 dk	Dürüstlük — sorulmadan söylenir

★ **5. bölüm neden kalbi:** Değerlendirmeci "mimariyi anlat" dediğinde çoğu aday katman isimlerini sayar. Sen bunun yerine **tek bir isteğin yolculuğunu** baştan sona anlatırsan, mimariyi *anlatmadan göstermiş* olursun.

2. Hangi bölüm rehberin neresinden anlatılıyor

Sunum bölümü	Rehberdeki kaynağı
Sistem nasıl çalışıyor	BÖLÜM 0 — iki bilgisayar, isteğin yolculuğu, teknoloji–adım tablosu
Karar çerçevesi	Giriş — üç ölçüt ve kararların dört kutusu
Stack çevirisi	A.1 ve A.2 tabloları
Sistem ne yapıyor	BÖLÜM B — on iki talep, her biri hangi teknolojiyle
Mimari ve katmanlar	E.1 modüler monolit + Clean Architecture
Teknoloji kartları	BÖLÜM C — 23 kart
Zor maddeler	E.2 SOLID · E.4 Factory · E.5 durum makinesi · E.6 mapping · E.7 hata yönetimi · E.8 eşzamanlılık
Test ve teslim	C.7 Testcontainers · C.11 Vitest · C.8 mimari testi · C.10 Docker
Dokümantasyon	E.12 ADR ve AI_USAGE
Reddedilen alternatifler	E.13 — Fastify, GraphQL, Repository, pg-boss
★ SLA politikası ve takvim	E.4 — süreler, çarpan, yüzdeler, karma takvim kararı
Veri modeli ve isimlendirme	E.9 — @map köprüsü, index, soft delete, audit, eşzamanlılık
Git akışı ve CI	E.11 — GitHub Flow, yedi kapı, GitLab farkı
Arayüz (UI) tarafının tamamı	BÖLÜM G — bir ekranın hayatı: sıra kararı (G.0), teknoloji katmanları (G.1), on adımlık akış (G.2)
Yapım sırası ve durum	BÖLÜM H — 17 adım, kutucuklarla ilerleme

Kod örneklerini sunumda kullanmak

Rehberdeki 71 kod bloğunun tamamı satır satır Türkçe açıklamalı. Kod okumayan biri de takip edebilir; bu yüzden ekranda **kodu gösterip yorumları okumak** mümkün.

Yorumlar önem sırasına göre işaretli — anlatırken bunlara odaklan:

İşaret	Ne demek	Sunumda
🚫	Yapılırsa sistem bozulur	"Bu satır olmazsa şifre özeti dışarı sızar"
⚠️	Kolayca gözden kaçır, sonucu ağır	"Bu hata tek kullanıcı testte hiç görünmez"
★	Tasarımın kalbi	"Kararın sebebi tam olarak burası"

★ **Soru geldiğinde işaretli satırı göster.** "Mükerrer bildirim nasıl engelliyorsun?" sorusuna, C.5'teki  işaretli iki satırı açmak, uzun uzun anlatmaktan daha inandırıcı.

★ Sunumda **rehberdeki sırayı takip etme**. Rehber referans kitabı; sunum ise bir anlatı. Sıra yukarıdaki akış tablosunda.

3. ★ Bir iş emrinin hayatı

Sunumun kalbi. Anlatım rehberde: **BÖLÜM F — Bir iş emrinin hayatı (sunucu tarafı)**.

★ **Arayüz (UI) tarafı için ikizi var: BÖLÜM G — Bir ekranın hayatı.** "Frontend'i anlat" sorusuna teknoloji listesi saymak yerine oradaki on adım anlatılır.

Her ikisi de **on adımlık** bir yolculuk ve her adımın hangi karara dayandığı tablosuyla birlikte orada. Bu iki bölümü anlatmak, mimariyi anlatmadan göstermek demektir — bu yüzden akışta toplam **8 dakika** ayrıldı (5 + 3).

4. Bilinen teknik borçlar

Rehberde: **KAPANIŞ → Bilinen teknik borçlar**. Sunumda sorulmadan söylenir.

5. Muhtemel sorular ve hazır cevaplar

Numaralar **rehberdeki** bölümleri gösteriyor.

Soru	Rehberde nerede
"Neden .NET değil?"	Giriş — üç ölçüt · A.1 hiçbir yeteneğin düşmediği tablo
"Neden Hangfire değil BullMQ?"	C.6
"Neden BullMQ, pg-boss değil?"	E.13 — yaygınlık ölçümüyle
"AutoMapper nerede?"	E.6 — aday kütüphanelerin ölçüm tablosu
"Repository Pattern neden yok?"	E.13
"Neden Next.js, Vite değil?"	C.2 — Next'in üç ürünün işini nasıl topladığı
"Next tek başına yetmez miydi?"	E.1 — dört koşul, üçü bu projede var
"Domain katmanı gerçekten bağımsız mı?"	C.8 — dependency-cruiser ; iddia değil CI kapısı
"Neden Redis eklendi?"	C.6 ve E.13 — kuyruk + hız sınırı + dağıtık kilit
"GraphQL düşündün mü?"	E.13 — izleme ve ön bellek gereksesiyle
"Fastify daha hızlı değil mi?"	E.13 — süre dağılımı tablosuyla
"Bu kadar test şart mı?"	Ödev §23 zorunlu tutuyor · C.7 sahte veritabanı neden yasak
"Servis yaşam döngülerini anlat"	C.1 §4 — Transient/Scoped/Singleton tablosu
"Eş zamanlı güncellemeyi nasıl çözdün?"	E.8 — version kolonu ve üretilen SQL · BÖLÜM F → 8. adımın açılımı — kolonun ne olduğu, iki kullanıcı tablosu
"Frontend'i anlat"	BÖLÜM G — on adımlık ekran akışı
"Önce backend mi yazdın, neden?"	G.0 — kural aslında "önce veri şekli kesinleşsin" + üç istisna
"Doğrulamayı neden iki kez yapıyorsun?"	BÖLÜM F → 1. adımın açılımı — curl örneğiyle
"N+1 problemi nedir, sende var mı?"	E.13 → REST/GraphQL — 51 sorgu vs 2 sorgu örneği
"Neden Fastify değil?"	E.13 — süre dağılımı + " <i>emeği index'lere harcadım</i> " açılımı
"Neden konteyner portları standart ama host portları farklı?"	C.10 → port ve konteyner planı — 12-factor, .env 'den gelen host portu
★ "SLA sürelerini neye göre belirlediniz?"	E.4 → " <i>Kararlaştırılan SLA politikası</i> " — dört öncelik, varlık kritikliği çarpanı, %50/%80 yüzdeleri
★ "Kritik arıza gece açılırsa ne oluyor?"	E.4 → karma takvim — Kritik 7/24, diğerleri mesai. Örnek: Cuma 16:00'da açılan bilet
★ "Kurumun isimlendirme kuralına nasıl uydunuz?"	E.9 → @map köprüsü — kodda dueAt , veritabanında due_at ; kurum kuralı değişirse yalnızca şema değişir

Soru	Rehberde nerede
"Mapping kütüphanesi yok demiştiniz, <code>@map</code> nedir?"	E.6 ve E.9 başındaki karşılıklı ayırım notu — biri nesne dönüşümü, diğeri ad eşlemesi
"Git akışınız nasıl?"	E.11 → GitHub Flow, dokuz adım İngilizce karşılıklarıyla
★ "GitLab'a geçerse ne olur?"	E.11 → CI adımları <code>pnpm ci:verify</code> 'de; iki platform dosyası da onu çağırıyor, ek iş çıkmaz
"CI'da tam olarak ne koşuyor?"	E.11 → yedi kapı tablosu (lint, tip, birim, entegrasyon, mimari, E2E, derleme)
"Aynı makinede iki proje çalışmaz mı?"	C.10 — konteyner içi sabit, host değişken; Testcontainers rastgele port

6. Anlatım sırasında dikkat

Her kararı kutusuyla birlikte söyle. Dört kutunun tanımı rehberin girişinde ("*Kararların dört kutusu*"). Dördüncü kutudaki her madde **ölçüyle** desteklenir — "*bence daha iyi*" denmez.

En çok soru gelecek üç yer:

Soru	Rehberde nerede
"AutoMapper nerede?"	E.6 — ölçüm tablosuyla
"Next tek başına yetmez miydi?"	E.1 — dört koşul
"Neden .NET değil?"	Giriş — üç ölçüt + yetenek tablosu

Bölüm 3'ü atlama. Akışta bu ikiliye toplam 8 dakika ayrıldı — tek başına en büyük pay; sebebi rehberin **BÖLÜM F** girişinde yazılı.

Bilmediğin bir şey sorulursa: "*Emin değilim, rehberde şu bölümde ayrıntısı var, bakıp deneyim.*" Uydurmaktan çok daha iyi karşılanır — rehber zaten ekranda açık olacak.

7. Rehber güncellenirken (kendime not)

Rehberin yazım kuralları kitte tanımlı: `11-agent-workflow.md` → "*Dışarıya giden doküman — anlatım standardı*". Özeti:

- Her kavram **üç adımda** açılır: gerçek hayat örneği → yazılımdaki tanımı → bu projede tam olarak nerede
- Kod görülmeden anlaşılmayacak her başlıkta kısa örnek olur
- Doküman, sahibinin bilgi seviyesini ele vermez
- Aynı gerekçe iki yerde yazılmaz — biri diğerine başlığıyla işaret eder

★ **Her session bittiğinde o session'ın kararları rehberde hemen işlenir.** En sona bırakılmaz; gerekçe, kararı verirken en net hatırlanır.